Nüwa: Enhancing Network Traffic Analysis With Pre-Trained Side-Channel Feature Imputation

Faqi Zhao, *Student Member, IEEE*, Wenhao Li, *Member, IEEE*, Huaifeng Bao, *Member, IEEE*, Zhaoxuan Li, *Member, IEEE*, Guoqiao Zhou, Wen Wang, and Feng Liu

***Abstract*—Network traffic classification stands as an essential endeavor within the realms of network security and management. The recent advances in learning-based methodologies have underscored their efficacy in deducing patterns from the side-channel features of encrypted network traffic. The unpredictability of traffic bursts can result in packet loss during retransmission, thereby generating fragmented feature patterns. Unfortunately, current approaches struggle to adapt to such fragmented features, often leading to a substantial decline in performance. To surmount this challenge, this paper introduces a pre-training-based framework, denoted as Nüwa, which imputes the side-channel features of encrypted network traffic, especially focusing on the temporal attributes of missing packets within a traffic session. Firstly, we propose a word-level Sequence2Embedding (S2E) module to transform side-channel features into tokens for model pre-training, as well as a Traffic Feature Masking strategy (TFM) to simulate the original flows changes in packet loss network. Besides, we also introduce a Traffic Feature Imputation (TFI) module to restore the missing values of original traffic flows in an efficient and context-aware manner. Experiments across four diverse real-world scenarios substantiate Nüwa's capacity to restore the performance of prevalent temporal models, while maintaining the integrity of the imputed features. Notably, Nüwa has also demonstrated an impressive resilience, even under conditions of extensive feature loss and domain adaptation. The Nüwa prototype has been made accessible to the public for further research and development (https://github.com/Timeless-zfqi/Nuwa).**

***Index Terms*—Traffic packet loss, side-channel feature, pre-training, feature imputation, domain adaption.**

I. Introduction

NETWORK Traffic Analysis and Classification (NTAC) has been widely used in cyberspace situational awareness and intrusion detection. In traditional network traffic classification tasks, researchers typically train models and perform classification using complete traffic features, such as FS-Net [1]. These approaches generally rely on the sufficiency and completeness of raw traffic data, with missing data handled through simple filling strategies (e.g., zero-padding or mean imputation). However, the escalating complexity of network environments and the information loss caused by packet drops pose significant challenges to traffic classification tasks [2]. With the surge in encrypted traffic, traditional rule-based methods [3] have become increasingly limited, which only focus on payload information.

Currently, the primary approach to NTAC tasks involves extracting side-channel features from raw traffic, categorized as feature engineering based methods. Despite their great performances, we have to figure out that they are inadequate for practical application, particularly in scenarios involving packet loss within network traffic, like wireless networks, subways, and etc. In other words, in practical network environments, these methods in the traffic domain fail to effectively address packet loss issues in traffic data. Therefore, how to effectively restoring the side-channel features of raw data with missing packets is crucial for NTAC tasks.

Network traffic side-channel feature imputation, aiming to recover missing packet information in traffic data, is an important technique in optimizing network management and bolstering security measures [4]. Traditional imputation methods for missing side-channel features basically rely on observed data, such as averages or nearest neighbor values, and techniques like Markov model [5]. However, these methods do not fully leverage session-wide information, thus suffering from the long-distance forgetting. These heuristic methods overlook the negative impacts of imputation on downstream tasks. Furthermore, in other fields, such as traffic data and image processing, researchers have proposed various advanced imputation methods to handle missing data. For instance, deep learning techniques like SAGCRN [6] can fill in missing parts by learning the spatiotemporal dependencies of the data. While imputation methods from non-traffic domains demonstrate excellent performance, they cannot be directly applied to restore missing traffic-related information due to the inevitable gaps between these tasks.

In recent years, pre-trained models have shown significant success in Natural Language Processing (NLP), utilizing large unlabeled datasets to learn unbiased logical relationships within texts. Given the temporal properties and contextual correlations inherent in traffic side-channel features, approaches like ET-BERT [7] and MAE [8] apply pre-trained models to

Received 4 June 2024; revised 21 December 2024 and 12 April 2025; accepted 9 June 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor S. Magnusson. Date of publication 3 July 2025; date of current version 18 December 2025. This work was supported by the National Natural Science Foundation of China (NSFC) under Grant E3101311F1. *(Corresponding author: Wen Wang.)*

Faqi Zhao and Zhaoxuan Li are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: zhaofaqi@iie.ac.cn; lizhaoxuan@iie.ac.cn).

Wenhao Li is with the Research Institute, China Telecom Company Ltd., Guangzhou 510660, China (e-mail: secliwenhao@163.com).

Huaifeng Bao is with Tencent Technology (Shenzhen) Company Ltd., Shenzhen 5180521, China (e-mail: finnbao@tencent.com).

Guoqiao Zhou, Wen Wang, and Feng Liu are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China (e-mail: wangwen@iie.ac.cn).

Digital Object Identifier 10.1109/TON.2025.3580557

2998-4157 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

directly process these features. They aim to allow the model to find an optimal solution for robust traffic representation. However, similar to de-duplication and denoising which are essential during the pre-processing stage, noise and information loss can also severely impact NTAC tasks.

We argue that in the field of traffic feature imputation, directly adopting an end-to-end pre-training models derived from NLP tasks will inevitably overlook the adverse effects of missing features on downstream classification models.

In this paper, we empirically verify the performance degradation issue of network traffic analyzers when dealing with incomplete side-channel features from high-performance networks. To solve this, we introduce the Nüwa[1] framework, which employs a pre-trained masking strategy for feature imputation. We first propose a raw traffic representation module named Sequence2Embedding (S2E), which transforms the datagram to language-like tokens for subsequent model pre-training. Thereinto, we also propose a Traffic Feature Masking (TFM) strategy to simulate network packet loss for pre-training Nüwa, as well as a Traffic Feature Imputation (TFI) module to impute the missing side-channel features and restore the raw data. This thus allows us to leverage the restored network traffic for downstream classification tasks. We evaluate six Deep Learning (DL) models using packet sequences for traffic classification of both source domains and source-free domains. Our experiments involve: i) empirical results of pre-training and feature imputation under both source domain and source-free domain; ii) comparison of Nüwa with traditional methods and models; iii) ablation studies on S2E, TFM, and TFI; and iv) performance analysis.

In summary, the contributions of this paper are summarized as follows:

- In this paper, we delve into the network traffic side-channel feature imputation, and empirically show an in-depth analysis revealing that mainstream DL models fail to achieve their purported performance when inferring from incomplete features in real-world network scenarios.
- We propose Nüwa for recovering missing side-channel features, including a Sequence2Embedding module for traffic token representation, a Traffic Feature Masking strategy for simulation of traffic feature changes caused by packet loss, and a Traffic Feature Imputation module for restoration of original flows' missing values for downstream NTAC tasks.
- Comprehensive experiments across four real-world scenarios confirm Nüwa's effectiveness and robustness in feature imputation, enabling SOTA classifiers to maintain performance despite network traffic packet loss.

II. Assumptions and Problem Formulation

A. Motivating Scenarios

Network traffic transmission faces various instability factors, such as network congestion and burst interruptions. These problems can lead to packet loss during transmission, which not only affects network performance but also significantly impacts traffic analysis-based detection systems, particularly in scenarios where side-channel features are used for security monitoring [9]. Our subsequent investigation (e.g., Section III) reveals that packet loss in different network environments is primarily caused by three phenomena, namely, complex topology, limited probe, and unpredictable network properties. However, in practice, existing DL-based detection solutions have been found to be heavily based on side-channel features and cannot support precise NTAC tasks. This means that missing side-channel features need to be restored to achieve accurate NTAC and network management. Alternatively, traditional imputation methods such as Markov are considered. The problem faced by this solution is that it does not utilize contextual information and still has limitations when dealing with high-dimensional and highly non-linear network data. More details about the disadvantages of common algorithms are discussed in Section VII.

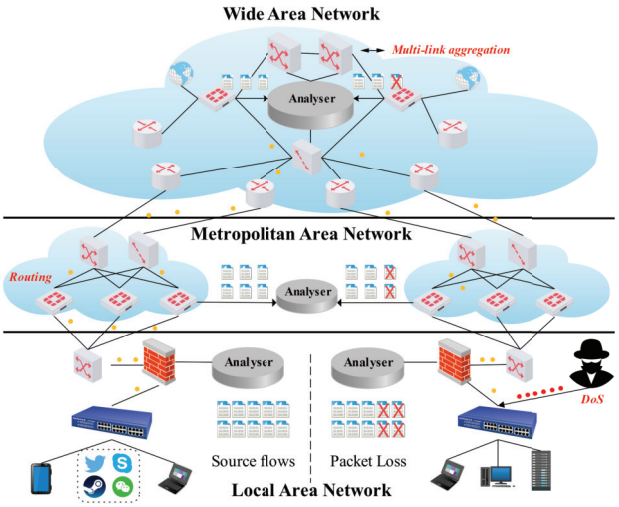


Fig. 1. Communication of packet loss scenarios. Packet loss in Local Area Networks (LAN), Metropolitan Area Networks (MAN), and Wide Area Networks (WAN) differs, with longer network links typically resulting in higher packet loss. In the MAN and WAN stages, any switch or router along the path may experience packet loss. Such process-related packet loss leads to incomplete traffic reaching the local analysis side, and on the LAN side, Denial of Service (DoS) attacks can also occur, making the valid data packets unreachable.

One positive aspect is that a new traffic side-channel feature imputation framework, called Nüwa, is introduced. It effectively combines traffic context and self-supervised training to simulate and learn from various packet loss scenarios. Specifically, the proposed Nüwa is able to address random packet loss, consecutive packet loss, and out-of-order packets through the TFM masking strategy. With these pre-training strategies, Nüwa only needs to be trained on large-scale unlabeled raw data. Such a new network traffic side-channel feature imputation methods is practical and feasible for in-deep network measurement.

B. Threat Model and Assumptions

Threat Model. The threat model considered in this work is illustrated in Fig. 1. We assume that traffic capture occurs

[1]The concept of Nüwa is inspired by an ancient Eastern myth of the creation of world. In the legend, Nüwa mends the sky with colorful stones, analogous to employing a pre-trained masking approach to impute for the missing side-channel features caused by packet loss.

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

within a Local Area Network (LAN) and has access to all packets received. Specifically, the capture mechanism can also be deployed on a network switch, enabling it to intercept packets exchanged between the client and server. In Fig. 1 (3-layer network), as concrete cases, packet loss often arises from complex network topologies, where multiple intermediate devices and redundant paths heighten the risk of dropped or rerouted packets, particularly during periods of network congestion or path reconfiguration. Secondly, limited probe where packets are dropped because intermediate routers or switches lack the buffer space to accommodate incoming traffic. Lastly, unpredictable network properties, such as varying link quality, fluctuating bandwidth, and network attack (Fig. 1 Dos) also contribute significantly to packet loss. Therefore, we primarily consider network packet loss phenomena and the out-of-order packets resulting from packets not being retransmitted in time.

Assumptions. In designing and evaluating the traffic feature imputation model based on the TFM strategy, we make the following assumptions to clearly define the scope of the research and the boundary conditions of the application scenarios:

Assumptions on Data Loss Characteristics. i) Packet loss typically exhibits local characteristics. Missing data points in sequences are not entirely random but tend to occur within specific time intervals or follow certain patterns (e.g., bursty packet loss). ii) Even with data loss, parts of the traffic sequence remain intact. The imputation model can utilize contextual information from these intact segments to infer and recover the missing data.

Assumptions on Noise Distribution. The TFM strategy generates mask data (e.g., random, Poisson-distributed, or locally concentrated missing positions) to approximate the characteristics of data loss observed in real-world networks. This assumption ensures that the masking strategies employed during the pre-training phase are aligned with realistic scenarios, allowing the imputation model to generalize effectively to real-world applications. Moreover, the noise mask distribution does not intentionally favor specific traffic patterns, thereby avoiding the introduction of biased assumptions during training and preserving the model's generalizability.

Assumptions on Traffic Feature Reliability. Despite packet loss, the quality of the remaining traffic feature values are reliable and have not been subject to additional tampering or malicious interference (such as data pollution or attacks). The pre-training task using the noise-based masking strategy (Masked Language Modeling, MLM) is consistent with the real-world traffic feature imputation task in terms of data distribution and missing patterns. Therefore, the representations learned by the imputation model during the pre-training phase can be effectively transferred to real-world imputation tasks.

C. Problem Formulation

In this section, addressing the aforementioned issues and assumptions, as shown in Table I, we formalize the problem as a traffic sequence imputation task. Given a traffic sequence $X = \{x_1, x_2, \ldots, x_n\}$, where, x_i corresponds to the value of the ith packet in the sequence, and $M = \{x_1, x_2, \ldots, x_n\} \in \{0, 1\}^n$ is the mask matrix indicating the positions of the missing data points in *X*, with $m_i = 0$ if x_i is missing and $m_i = 1$ otherwise, and each element $m_i \in M$ corresponds to a specific position in *X*. We are given an unlabeled training set $S_{train} = (X_i, M_i)_{i=1}^{N}$ containing pairs of traffic sequences and corresponding mask matrices.

TABLE I
THE SYMBOL DESCRIPTION FOR PROBLEM FORMULATION

Notation	Definition
X	The traffic side-channel feature
M	The matrix of the missing data positions
$\hat{X}$	The traffic sequence after feature imputation
S_{train}	The training set
S_{test}	The testing set
f_θ	A trained-well feature imputation model
$\mathcal{L}_{\text{MLM}}$	The total prediction loss of f_θ
$\ell(x_i, \hat{x}_i)$	A loss function of x_i and $\hat{x}_i$
$\mathbb{E}_{(X,M)}$	The expectations of X and M

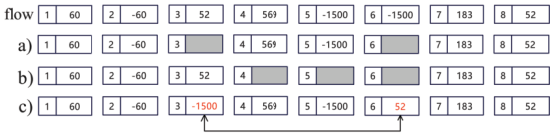


Fig. 2. Network packet loss and out-of-order.

The imputation model f_θ is pre-trained on S_{train}. After pre-training, the model f_θ is used to impute the missing data and generate the completed traffic sequence $\hat{X} = \hat{x_1}, \hat{x_2}, \hat{x_3}, \ldots, \hat{x_n}$ (for all $m_i = 1$, $\hat{x} = x_i$, and for all $m_i = 0$, infer $\hat{x}_i$ such that $\hat{x}_i \approx x_i$).

Based on the above task, we define a feature imputation framework named Nüwa. The goal is to optimize the imputation function $f_\theta : (X, M) \rightarrow \hat{X}$. The framework is designed to perform optimally in the following three stages:

i) **Pre-training Based on TFM Strategy**: By introducing different types of masking strategies TFM, we simulate packet loss patterns in real networks, including:
- Random loss of individual packets: random distribution $m_i = 0$, Fig. 2 a).
- Consecutive loss of multiple packets: $m_{i:i+k} = 0, k \geq 1$, Fig. 2 b)
- Out-of-order packets: $m_i \rightleftharpoons m_j$, Fig. 2 c).

The MLM task is used to train f_θ to minimize the loss:

$$\mathcal{L}_{\text{MLM}} = \mathbb{E}_{(X,M)} \left[\sum_{i:M_i=0} \ell\left(x_i, \hat{x}_i\right) \right] \tag{1}$$

where $\ell(.,.)$ is the loss function and $\mathbb{E}_{(X,M)}$ denotes the expectation with respect to the joint distribution of *X* and its corresponding matrix *M*.

ii) **Source Domain Adaptation**: Given a prior dataset S^A_{train}, the sequences to be imputed are derived from Scenario A. The trained imputation model is then applied to impute the traffic sequences X^A in S^A_{test}, generating the restored sequences $\hat{X}^A$:

$$\hat{X}^A = f_\theta(X^A, M^A) \tag{2}$$

iii) **Source-free Domain Adaptation**: Given a prior S^A_{train}, the sequences to be imputed come from scenario B. The trained imputation model is used to impute the

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

traffic sequences X^B in S^B_{test}, generating the complete sequences $\hat{X}^B$.

Essentially, the primary challenges we encounter include the diversity of pre-training packet loss strategies, domain adaptation in the absence of source domain data, and the successful adaptation to downstream tasks. The completed sequences $\hat{X}$ must remain efficient for classification or detection tasks. Through this formalization, we propose a unified framework that addresses the requirements of traffic side-channel feature imputation in complex network environments (e.g., Section IV).

III. Measurement Study

In this section, to better illustrate the effects of traffic side-channel feature loss on existing models, we systematically evaluate the performance of six representative deep learning models on real-world and simulated network traffic with packet loss. The models in this paper employs traditional Convolutional Neural Network (CNN), Long-Short Term Memory (LSTM), GRU (Gated Recurrent Unit) [3], Denoising Autoencoder (SDAE) [10]algorithm, and advanced Time Series Transformer (TST) [11], FS-Net [1] algorithm for validation. We conducted experiments in two environments: real network packet loss and simulated packet loss, and we apply two metrics, namely accuracy (Acc) and F1-Score (F1) to evaluate the traffic classification results.

We conducted experiments across various real-world network traffic scenarios, including website traffic via DoH (DNS over HTTPS), IoT traffic for small devices, mobile application traffic for mobile devices, and PC terminal traffic.

Scenario d_1 involves traffic generated by mobile applications. We selected the CIC-AndAdw-2017 dataset [12], which was obtained by installing Android applications on real devices and capturing their network traffic. The dataset is generated from 1,900 applications with the following three categories: Adware (250 apps, 13345 sessions), General Malware (150 apps, 8673 sessions) and Benign (1500 apps, 29710 sessions). Specially, the network traffic of both the malware and benign (20% malware and 80% benign).

Scenario d_2 is website DoH network traffic, using the CIRA-CIC-DoHBrw-2020 dataset [13], which provides for analyzing, testing, and evaluating DoH traffic within covert channels and tunnels. DoH was obtained by accessing the top 10k Alexa websites, using browsers and DNS tunneling tools that support the DoH protocol. It contains Non-DoH (browsing thousands of websites from Alexa domain), Benign-DoH (using Mozilla Firefox and Google Chrome web browsers) and Malicious-DoH (DNS2TCP, DNScat2, Iodine, generated by DNS tunnel tools) with 193,883 sessions.

Scenario d_3 is a real-time dataset and benchmark for large-scale attacks in IoT environment [14], using an extensive topology compose of 105 real IoT devices and adopting IoT devices as attackers and victims. It's divided into 7 classes against IoT devices and demonstrated how they can be reproduced. In this study, we focus on six types of TCP-based attacks, including DoS-HTTP-Flood, DDoS-HTTP-Flood, DoS-TCP-Flood, MITM-Spoofing, DDoS-TCP-Flood, and DNS-Spoofing with 197,888 sessions.

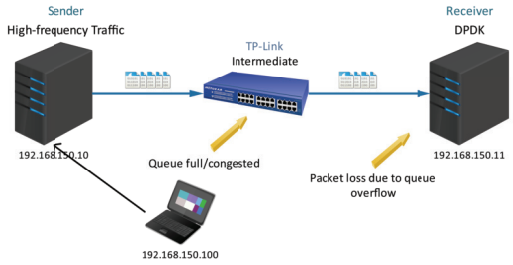


Fig. 3. Real-world topological diagram.

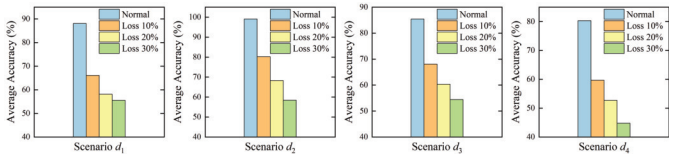


Fig. 4. Classification results of average accuracy with packet loss in simulated d_1, d_2, d_3 and d_4.

Scenario d_4 is generated by PC terminal devices using the USTC-TFC dataset [15], which contains 10 types of benign and 10 types of malicious application traffic. However, only six application flows with 138,901 sessions were selected due to some being predominantly or exclusively UDP flows.

A. Experiments With Simulated Traffic Packet Loss

1) Experiment Setup: **Simulating Traffic Packet Loss.** To emulate real-world conditions in our testbed, we introduce packet loss using both random and Poisson distributions, removing 10%, 20%, and 30% of packets from the traffic. In different network segments and environments, the packet loss rate varies. In typical cases, network packet loss is below 10%, while in MAN, the packet loss rate can range from 10% to 20% [16]. To account for extreme conditions, we also set the packet loss rate to 30% in our experiments to better perform stress testing. The raw traffic is pre-processed to isolate TCP flows, which are then segmented into individual sessions. For each session, bi-directional packet length features are extracted to serve as the basis for analysis.

Original Flow Data Processing. We cap the maximum packet length at 1600 bytes. This decision is based on the Ethernet protocol (RFC 894), which is widely used for transmission and defines the maximum transmission unit (MTU) as 1500 bytes, including both the payload and the packet header. However, considering the variability introduced by local area network (LAN) configurations and potential MTU adjustments during internal and external network transitions, we extend the threshold slightly to accommodate such variations, setting the maximum packet length to 1600 bytes. Furthermore, based on practical considerations and the handshake and retransmission mechanisms inherent to TCP, we define the minimum session length as 20 packets. To simulate real-world network conditions, we introduce random packet loss by applying masks ranging from 10% to 30%.

Model Training and Testing. In the initial training phase across environments d_1, d_2, d_3, and d_4, we adhere to an 8:2 split for training and testing flows. Subsequently, we assess all models using packet length sequences with both

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

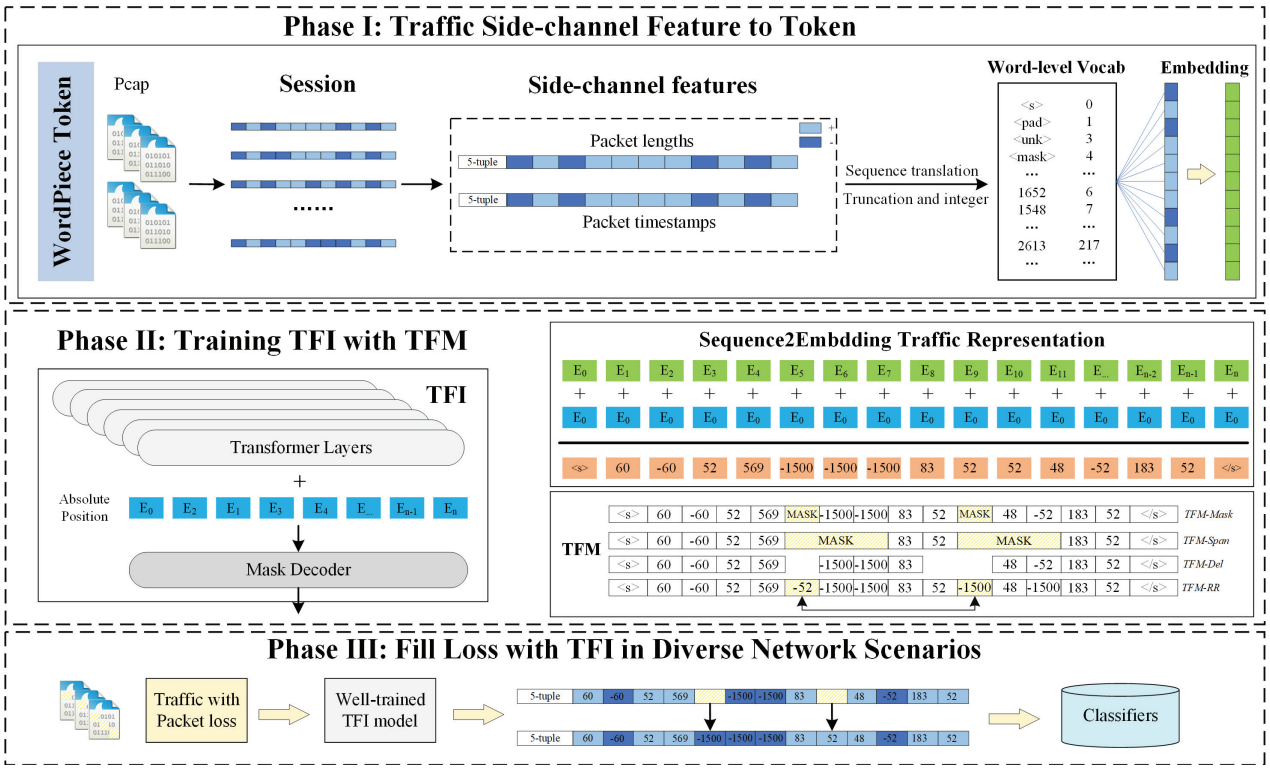


Fig. 5. Overview of Nüwa. Phase I involves extracting sequence features from raw traffic to construct a world-level vocabulary, which is then encoded into tokens. Phase II (right) involves S2E Traffic Representation, where the encoded sequences are augmented with positional information to form the input for the pre-trained TFI model. The TFM is designed with four pre-training strategies to simulate packet loss and compute pre-training loss. Phase II (left) represents our side-channel feature imputation model, which uses S2E and TFM for pre-training. Finally, Phase III involves restoring traffic with packet loss through the well-trained TFI and passing the restored data downstream for classification.

TABLE II

RESULTS WITH REAL-WORD SCENARIO d_2 WITH PACKET LOSS

Model	Normal		Loss 10%		Loss 20%		Loss 30%	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1
CNN	99.86%	99.86%	70.46%(29.40%↓)	70.48%(29.38%↓)	44.01%(55.85%↓)	42.47%(57.39%↓)	30.58%(69.28%↓)	25.90%(73.96%↓)
LSTM	98.83%	98.83%	57.14%(41.69%↓)	56.32%(42.51%↓)	38.69%(60.14%↓)	37.32%(61.51%↓)	30.23%(68.60%↓)	27.73%(71.10%↓)
GRU	99.22%	99.22%	57.14%(42.08%↓)	56.56%(42.66%↓)	38.90%(60.32%↓)	37.38%(61.84%↓)	29.25%(69.97%↓)	27.27%(71.95%↓)
TST	99.20%	99.20%	84.3%(14.90%↓)	84.64%(14.56%↓)	63.09%(36.11%↓)	63.99%(35.21%↓)	40.32%(58.88%↓)	37.90%(61.30%↓)
SDAE	94.84%	94.84%	52.93%(41.91%↓)	52.94%(41.90%↓)	38.47%(56.37%↓)	38.47%(56.37%↓)	32.64%(62.20%↓)	32.09%(62.75%↓)
FS-Net	99.78%	99.78%	96.30%(3.48%↓)	96.32%(3.46%↓)	85.10%(14.68%↓)	85.34%(14.44%↓)	70.43%(29.35%↓)	70.78%(29.00%↓)
Average	**98.62%**	**98.62%**	**98.62%(28.91%↓)**	**98.62%(29.08%↓)**	**69.71%(47.24%↓)**	**69.54%(47.79%↓)**	**51.38%(59.71%↓)**	**50.83%(61.67%↓)**

TABLE III

RMSE RESULTS OF REAL-WORD AND SIMULATED SCENARIOS

Scenarios	Simulated			Real-word		
	10%	20%	30%	10%	20%	30%
d_1	0.1596	0.1924	0.2102	0.0445	0.1076	0.1716
d_2	0.0736	0.0938	0.1103	0.0523	0.0986	0.1428
d_3	0.1403	0.1833	0.2111	0.0746	0.1082	0.1793
d_4	0.1076	0.1344	0.1543	0.0305	0.0752	0.1260

intact and missing features. The input sequence length is standardized. And the majority of session lengths (512 packets and below) accounted for over 95% of the total sessions. Please refer to Fig. 13 in Section A of [17] for detailed results. Although the proportion of longer sessions increased slightly, the exponentially growing training overhead associated with them is unacceptable. Therefore, in line with the time-series-based NTAC research discussed in Section VII, sequences longer than 512 packets are truncated to maintain efficiency while still capturing the relevant data from different scenarios.

2) Experiment Results: We conducted packet loss experiments across four scenarios, d_1, d_2, d_3, and d_4 to simulate packet loss. Table VIII shows that the six DL models achieve over 80% accuracy and F1 for intact flow data across network conditions d_1 to d_4. However, performance declines as packet loss increases, particularly in d_2, where CNN's F1 drops from 99.30% to 34.96% when packet loss reaches 30%. Accuracy and F1 for all models decrease by 18.86% to 40.67% and 19.06% to 41.01%, respectively, under d_2's 10% to 30% packet loss. As depicted in Fig. 4, the results from the traffic packet loss experiments across the four aforementioned scenarios reveal varying levels of performance degradation in all the DL models examined. Similar trends are observed in other scenarios. Please refer to Table VI-X in online version Section A of [17] for detailed results.

B. Experiments With Real-World Traffic Packet Loss

1) Experiment Setup: **Replayed traffic with packet loss.** As shown in Fig. 3, our physical testbed has a 10 Gbps

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

CLASSIFICATION RESULTS OF REAL-WORD SCENARIO d_2 WITH IMPUTATION LOSS SIDE-CHANNEL FEATURES

Model	Imputed Loss 10%		Imputed Loss 20%		Imputed Loss 30%	
	Acc	F1	Acc	F1	Acc	F1
CNN+TFI	98.15%(27.69%↑)	98.15%(27.67%↑)	92.76%(54.14%↑)	92.68%(55.68%↑)	84.08%(67.57%↑)	83.73%(72.25%↑)
LSTM+TFI	94.91%(37.77%↑)	94.87%(38.55%↑)	88.08%(56.22%↑)	87.79%(57.55%↑)	81.59%(64.68%↑)	83.99%(67.14%↑)
GRU+TFI	95.05%(37.91%↑)	95.02%(38.46%↑)	87.35%(56.15%↑)	87.06%(57.64%↑)	79.66%(65.80%↑)	78.94%(67.75%↑)
TST+TFI	98.49%(14.19%↑)	98.48%(13.84%↑)	95.25%(35.4%↑)	95.23%(34.49%↑)	83.90%(58.17%↑)	83.45%(60.58%↑)
SDAE+TFI	91.83%(38.90%↑)	91.77%(38.83%↑)	85.93%(53.36%↑)	85.63%(53.30%↑)	78.70%(59.19%↑)	78.02%(59.68%↑)
FS-Net+TFI	99.49%(3.19%↑)	99.49%(3.17%↑)	98.79%(14.39%↑)	98.78%(14.15%↑)	95.34%(29.06%↑)	95.43%(28.71%↑)
Average	**96.32%(26.61%↑)**	**96.30%(26.76%↑)**	**91.36%(44.94%↑)**	**91.20%(45.47%↑)**	**83.88%(57.41%↑)**	**83.93%(59.35%↑)**

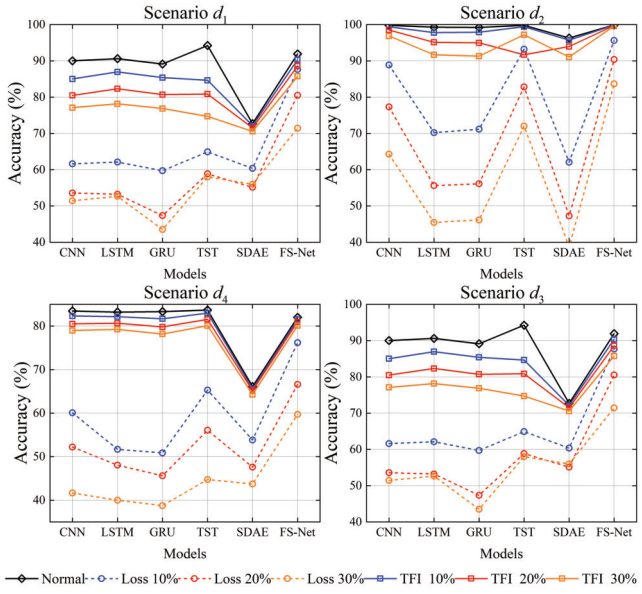


Fig. 6. Classification results of packet with intact, loss and imputed loss in 4 scenarios.

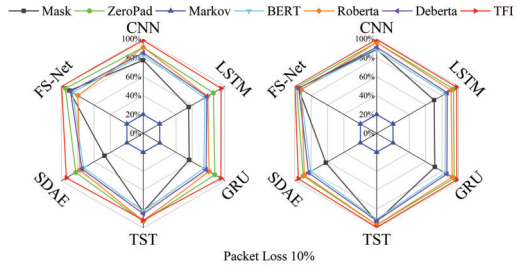


Fig. 7. Classification results with different feature imputation methods in d_1, d_2.

TABLE V

RMSE RESULTS OF S2E

Module	Imputed 10%	Imputed 20%	Imputed 30%
w/o S2E	0.1524	0.1668	0.1770
w/o None-S2E	0.7903	0.8272	0.8531
w/o BPE	0.1736	0.1943	0.2107

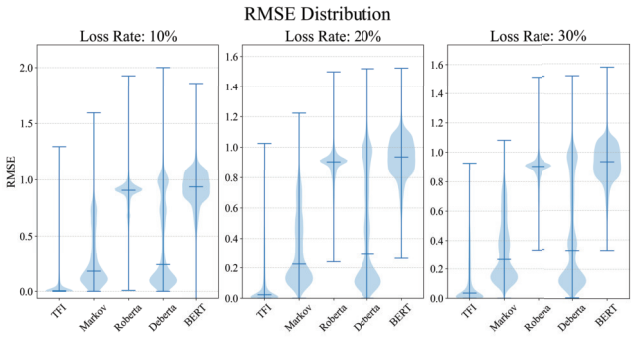


Fig. 8. RMSE distribution on different feature imputation in d_2.

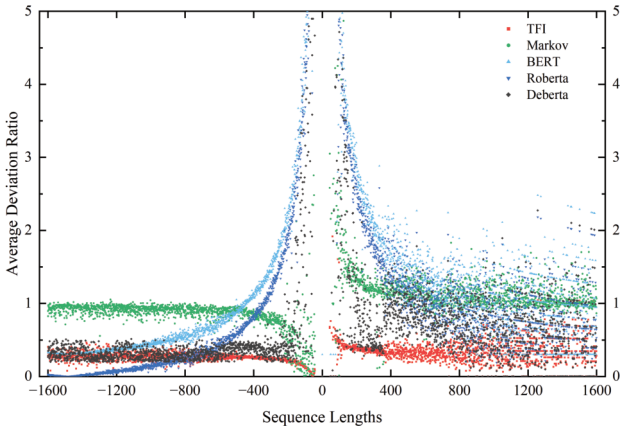


Fig. 9. Average-deviation-ratio.

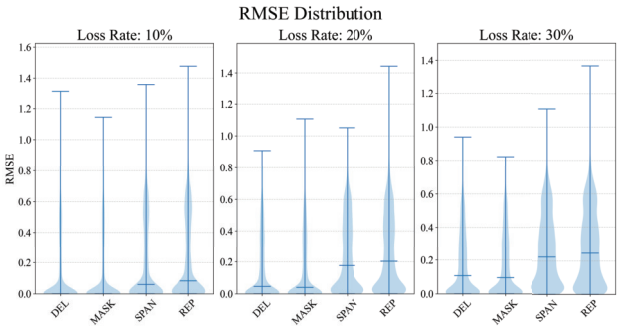


Fig. 10. RMSE of various TFM sub-strategies.

dataplane between two hosts: one serves as a packet generator and the other is used to receive packets. The packet generator is equipped with two 10 GbE Intel X722 network cards, an Intel Intel Xeon Gold 5218, and 256 GB memory. The second host has the same configuration as the first, except only a 10 GbE Intel X722 network card. Both hosts are running CentOS 8.2. Multiple 10 GbE TP-Link TL-SH1452 switches (48 ports with10/100/1000M) are used to connect two hosts and build various topology structures and the CPU transmit and receive limits of the switch are key to

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

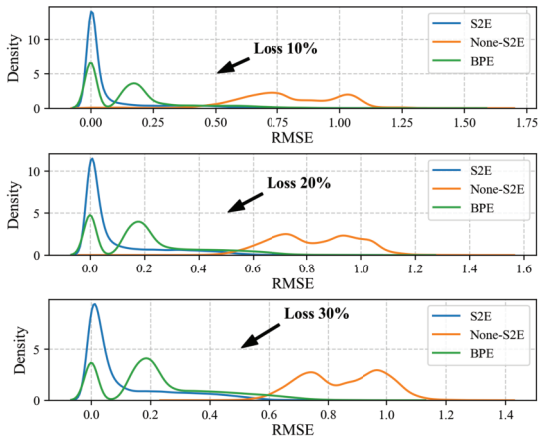


Fig. 11. Contribution of the S2E module compared with None-S2E and BPE.

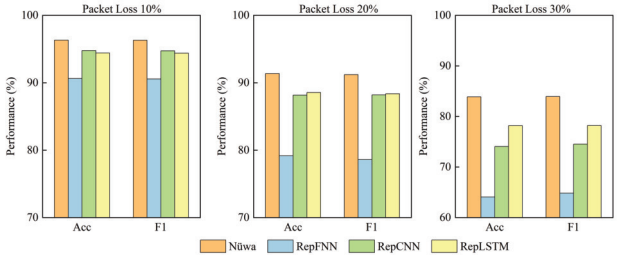


Fig. 12. Average accuracy with different main structure.

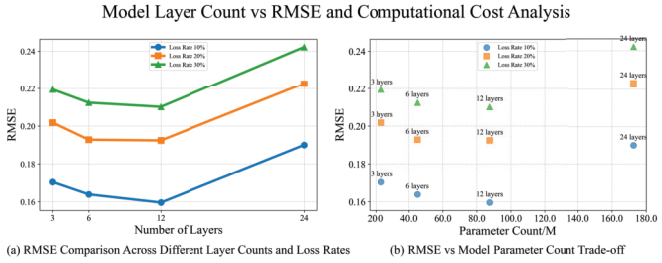


Fig. 13. Contribution of the TFI module with different layers.

TABLE VI

FEATURE IMPUTATION SPEED OF NÜWA IN DIFFERENT SCENARIOS

Speed (samples/s)	Imputed Loss 10%	Imputed Loss 20%	Imputed Loss 30%	Parameters
d_1	58	54	50	44.99M
d_2	49	47	45	45.15M
d_3	70	65	62	45.15M
d_4	76	69	65	45.10M
Average	**63**	**58**	**55**	/

TABLE VII

FEATURE IMPUTATION SPEED OF NÜWA IN DIFFERENT NETWORK LAYERS

Speed (samples/s)	Imputed Loss 10%	Imputed Loss 20%	Imputed Loss 30%	Parameters
3 layer	91	74	65	23.75M
6 layer	58	54	50	44.99M
12 layer	29	28	27	87.51M
24 layer	16	15	15	172.56M

simulating bandwidth overflow. We use pktgen-dpdk 23.07 for traffic replay. We conducted stress tests on the forwarding capabilities of Gigabit Ethernet port. At a transmission rate of 1 Gbps, the maximum receiving rate on the receiving end was approximately 962 Mbps. Therefore, we controlled the packet loss rate by gradually increasing the transmission rate.

Original Flow Data Processing. Initially, we begin by conducting experiments on four widely recognized traffic datasets commonly used in DL-based traffic classification. Next, we replay these flows at varying rates to induce controlled packet loss, measured as the difference between the number of packets sent and received. Finally, we perform experiments across the four scenarios described earlier to evaluate performance under different network conditions.

Model Training and Testing. To evaluate the performance of NTAC models on network traffic with packet loss in real-world network environments, we applied the same six DL models to classify flows affected by 10% to 30% packet loss. The experimental setup mirrored that used for simulated traffic packet loss, with session lengths ranging from 20 to 512 packets and a maximum packet length of 1600 bytes. Subsequently, we conducted experiments across the four scenarios previously described to assess model robustness under varying conditions.

2) Experiment Results: We conducted packet loss experiments in four real-world network scenarios. Taking scenario d_2 as an example, as shown in Table II, the table shows that each DL model achieved an accuracy of over 94% before any packet loss occurred. However, when applying these well-trained models in packet loss environments (i.e., 10% to 30%), similar with simulated randomized environment, we observe a significant drop in their performance. For example, the F1 of CNN decreases from 99.86% in a normal network environment to 70.48% with 10% packet loss and further drops to 25.90% with 30% packet loss. We can see that the average accuracy of all the models decreases by 28.91% at least to 59.71% at most in the network environments 10% to 30%, and the average decreases by 29.08% at least and 61.67% at most. The experimental results in Table XI-XIII with the replayed flows from d_1,d_3,d_4 demonstrate similar results. Please refer to Section A of [17] for detailed results. The results demonstrate that existing deep learning models fail to enable robust NTAC in diverse packet loss network environments.As depicted in Fig. 4, the results from the traffic packet loss experiments across the four aforementioned scenarios reveal varying levels of performance degradation in all the DL models examined.

C. In-Depth Analysis

After comparing the performance of the deep learning models under packet loss across diverse network environments, we observe significant variations in performance. The poor results can be attributed to the models' inability to account for the drastic changes induced by large packet losses, despite the fact that the traffic flows in different network environments are subject to the same packet loss rates. This suggests that the models lack sufficient robustness to handle the impact of severe packet loss, resulting in performance discrepancies across different environments.

High packet loss significantly changes traffic patterns, hindering classification. ML and DL-based detection methods, dependent on intact side-channel features, struggle with real-world traffic loss. Yet, packet length omissions follow TCP

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

protocols, allowing for robust imputation of missing side-channel features across various network conditions.

IV. System Design of Nüwa

Real-world network are dynamic and unpredictable, with factors like traffic bursts causing significant variability. We require restore the original side-channel features to reduce the impact of packet loss on existing DL models by Nüwa.

A. Overview

We propose a network traffic side-channel feature imputation framework, termed Nüwa, which employs a pre-trained noise masking approach. Nüwa utilizes a Transformer-based neural network architecture comprising three modules: a Sequence2Embedding (S2E) for traffic representation, a Traffic Feature Masking strategy (TFM), and a Traffic Feature Imputation model (TFI). The framework's architecture is depicted in Fig. 5. The first module is used to transform side-channel features into tokens for pre-training. TFM can help us to obtain a number of new flows that simulate how the original flows change in packet loss network. And the missing values are imputed by TFI to restore the original flows.

Nüwa aims to maintain the performance of DL models in real network with packet loss by learning a generic representation of encrypted traffic's side-channel features and restoring them across various scenarios, including Website, IoT, Mobile Application, and PC Terminal traffic. Without requiring traffic labels for specific scenarios or prior network knowledge, the Nüwa framework consists of two stages: (1) pre-training, where TFM strategy is applied to learn application side-channel feature patterns from a large unlabeled dataset, and (2) feature imputation, where imputation model predicts and fills in missing values within input sequences, enabling the restoration of intact features for downstream tasks.

B. Sequence2Embedding Module

In the realm of NLP, pre-trained language models such as RoBERTa [18] and DeBERTa [19] commonly employ the Byte Pair Encoding (BPE) [18] method for sentence tokenization. However, for traffic side-channel features, the BPE's fragments often lack semantic meaning. For example, the sequence s = [52, -44, 40, ..., -1500, 40] tokenized by BPE results in s_1 = [52, 44, 40, ..., 00] and combined sub-words s_2 = [5200, 521, 441, ..., 5215]. This paper introduces: (1) a *WordPiece token generator* for encoding packet length and timestamp sequences, and (2) a *Token2Embedding* that uses content and position embeddings to enhance temporal and content relationships in input sequences.

WordPiece token generator. We introduce a word-level token generator applicable to a variety of features. For the packet length feature, each value is treated as a distinct word within the range (-1600, 1600). We scale the values of the traffic packet length features to the interval [0, 3200] and derive the vocabulary from the raw data. As depicted in Fig. 5 Phase I, for the packet arrival time feature, timestamps are amplified by a factor of $1e4$ and then truncated to integers to establish a timestamp vocabulary with a range of [0, 10000].

Token2Embedding. Traffic sequence features are uniquely defined by packet direction, size, and arrival time. The relative positioning of packets within flows and sessions is fixed. For example, the TCP protocol's Three-way Handshake and Four-Way Wavehand are sequential patterns. Moreover, application traffic features may show periodicity, such as in the website sequence [52, -52, 40, 457, -40, -1500, -1500, 40, -1500, -1500, 40, -1500, -1500, -1500, -1500, 40, ..., -40, 40, 40, -40]. Here, both the initial [40, -1500, -1500][1] and the later [40, -1500, -1500][2] are contextually similar, but have different arrival times, indicating that the absolute position of a masked word in the packet sequence is significant.

Each token from S2E is represented by two embeddings: content and position. Unlike BERT [7] and ET-BERT [20], which sum different embedding types for the final representation, we omit SEP and NSP, treating the sequence as an intact sentence. We introduce special tokens [PAD], [s], [/s], [UNK], and [MASK] with our *WordPiece token generator*. Each input sequence is standardized to 514 tokens, initiated by [s], terminated by [/s], and padded to 512 feature tokens using [PAD]. The embedding for each token, $S^I_{i,j}$, jointly represents its content (packet size, direction) and relative position derived from the packet's timestamp. In the *WordPiece token generator*, $S^I_{i,j}$ is represented by two vectors for content and position, and attention weights are calculated using disentangled matrices from the content vector $\boldsymbol{S}_i$ and relative position vector $\boldsymbol{P}_{i|j}$.

$$\begin{aligned} S^I_{i,j} &= \left\langle \boldsymbol{S_i}, \boldsymbol{P_{i|j}} \right\rangle \times \left\langle \boldsymbol{S_j}, \boldsymbol{P_{j|i}} \right\rangle^\top \\ &= \boldsymbol{S_i S_j^\top} + \boldsymbol{S_i P_{j|i}^\top} + \boldsymbol{P_{i|j} S_j^\top} + \boldsymbol{P_{i|j} P_{j|i}^\top} \end{aligned} \tag{3}$$

As Eq. (3) shows, in the disentangled attention, $\boldsymbol{S_i P_{j|i}^\top}$ and $\boldsymbol{P_{i|j} S_j^\top}$ enhance the relative relationship between content-position and position-content, respectively. $\boldsymbol{S_i S_j^\top}$ enhance the absolute relationship between content-content and position-position absolute relationships. However, $\boldsymbol{P_{i|j} P_{j|i}^\top}$ position-to-position term does not provide much additional information and is removed from Eq. (3) in our implementation.

The standard self-attention operation can be formulated as:

$$\begin{aligned} &\boldsymbol{Q} = \boldsymbol{S W_q}, \boldsymbol{K} = \boldsymbol{S W_k}, \boldsymbol{V} = \boldsymbol{S W_v}, \boldsymbol{A} = \frac{\boldsymbol{Q K^\top}}{\sqrt{d}} \\ &\boldsymbol{S_o} = \mathrm{softmax}(\boldsymbol{A}) \boldsymbol{V} \end{aligned} \tag{4}$$

where $\boldsymbol{S} \in R^{N \times d}$ represents the input hidden vectors, $\boldsymbol{S_o} \in R^{N \times d}$ the output of self-attention, $\boldsymbol{W_q}$, $\boldsymbol{W_k}$, $\boldsymbol{W_v}$ are the projection matrices, $\boldsymbol{A} \in R^{N \times N}$ is the attention matrix, N the length of the input sequence, and d the dimension of hidden states.

Denote k as the maximum relative distance, $\ell(i,j) \in [0, 2k)$ as the relative distance from token i to token j, which is defined as [19]:

$$\ell(i,j) = \begin{cases} 0, & i - j \le -k \\ 2k - 1, & i - j > k \\ i - j + k, & others. \end{cases} \tag{5}$$

And the disentangled self-attention with relative position bias as Eq. (6), where $\boldsymbol{Q}_c$, $\boldsymbol{k}_c$ and $\boldsymbol{V}_c$ are the projected content

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

vectors generated using projection matrices, $\boldsymbol{W}_{q,c}$, $\boldsymbol{W}_{k,c}$ and $\boldsymbol{W}_{v,c} \in R^{d\times d}$ respectively, $\boldsymbol{P} \in R^{2k\times d}$ represents the relative position embedding vectors shared across all layers, and $\boldsymbol{Q_r}$ and $\boldsymbol{K_r}$ are projected relative position vectors generated using projection matrices $\boldsymbol{W_{q,r}}$ and $\boldsymbol{W_{k,r}} \in R^{d\times d}$ respectively.

$$\begin{aligned}
&\boldsymbol{Q_c} = \boldsymbol{SW_{q,c}}, \boldsymbol{K_c} = \boldsymbol{SW_{k,c}}, \boldsymbol{V_c} = \boldsymbol{SW_{v,c}},\\
&\boldsymbol{Q_r} = \boldsymbol{PW_{q,r}}, \boldsymbol{K_r} = \boldsymbol{PW_{k,r}}\\
&\tilde{S}^I_{i,j} = \boldsymbol{Q^c_i K^{c\top}_j} + \boldsymbol{Q^c_i K^r_{\ell(i,j)}} + \boldsymbol{K^c_j Q^r_{\ell(j,i)}}^{\top}\\
&\boldsymbol{S_o} = \text{softmax}\left(\frac{\tilde{S}^I}{\sqrt{3d}}\right)\boldsymbol{V_c} \qquad (6)
\end{aligned}$$

$\tilde{S}^I_{i,j}$ is the element of attention matrix $\tilde{\boldsymbol{S}}^I$, representing the attention score from token i to token j. $\boldsymbol{Q^c_i}$ and $\boldsymbol{K^c_j}$ is the i-th row of $\boldsymbol{Q}_c$ and j-th $\boldsymbol{K}_c$. $\boldsymbol{Q}^r_{\ell(j,i)}$ is the $\ell(i,j)$-th row of $\boldsymbol{Q}_r$ with regarding to relative distance $\ell(j,i)$. The position-to-content term is calculated as $\boldsymbol{K^c_j Q^r_{\ell(j,i)}}$. The content-to-position term is calculated in a similar way.

Finally, a scaling factor of $\frac{1}{\sqrt{3d}}$ is applied on $\tilde{\boldsymbol{S}}^I$. S2E module captures the relative positions in all the Transformer layers and only uses absolute positions as complementary information when decoding the masked words.

C. Traffic Noising Based Self-Supervised Pre-Training Mask Strategy

The pre-training model, akin to BERT, employs a mask strategy where 10% of the mask tokens remain unchanged, 10% are substituted with random tokens, and 80% are replaced by [MASK] tokens. RoBERTa and DeBERTa adopt this approach but also implement a dynamic masking strategy where the masked words within a sentence change with each iteration.

We contend that a dynamic mask strategy alone is insufficient to capture the true characteristics of network traffic packet transmission, which includes the loss of side-channel features due to packet loss, as well as potential delays and out-of-order issues at each network node. In this paper, we introduce the TFM strategy, as illustrated in Fig. 5 Phase II *TFM*, encompassing Token Mask, Token Deletion, Span Mask, Random Replace, and Unchanged.

Token Mask. In instances of discrete packet loss, where the sequence exhibits non-continuous gaps, a random selection of tokens from the original sequence is replaced with the special [MASK] token. These masked tokens are subsequently predicted and imputed by the TFI.

Token Deletion. Tokens are randomly removed from the input sequence, which may be consecutive or non-consecutive.

Span Mask. In scenarios where network disruptions lead to the loss of multiple consecutive packets, we replace multiple consecutive sub-sequences within the original sequence with a unique [MASK] token. Each continuous sub-sequence follows a Poisson distribution with $\lambda = 3$. This masking strategy is inspired by SpanBERT, which samples text fragment lengths based on a geometric distribution and replaces these fragments with sequences of [MASK] tokens of the same length. Unlike SpanBERT, our approach involves multiple samplings from a Poisson distribution to represent sub-sequences of varying lengths, allowing the model to learn the quantity of missing tokens in a traffic sequence through Span Mask.

Random Replace. To address the packet arrival out-of-order issue, two positions within the sequence are randomly selected and their values are swapped.

Unchanged. The position subjected to random masking retains its original value unaltered.

Therefore, the pre-training loss L of the TFI is:

$$L = L_{Mask} + L_{Span} + L_{RR} \qquad (7)$$

D. Traffic Side-Channel Feature Imputation Model

In NLP, Transformer-based pre-training architectures excel at fill-in-the-blank tasks. Large-scale pre-trained language models use MLM to learn contextual word representations from vast text corpora, where the model predicts masked words based on surrounding context. As depicted in Fig. 5 Phase II *TFI*, we employ a 12-layer Transformer encoder for the pre-training framework. Specifically, given a sequence $\boldsymbol{S}^I = \{x_1, X_2, X_3, \ldots, X_i\}$, during training, 15% of the tokens in the input text are masked by the TFM strategy, resulting in $\tilde{\boldsymbol{S}}^I$. The model predicts original tokens from surrounding context in an unsupervised pre-training process, learning rich contextual representations of traffic. In the MLM task, masked tokens are replaced by the [MASK] token, prompting the model to predict them. The model calculates the probability distribution across the entire vocabulary for each masked token and chooses the token with the highest probability as its prediction. As shown in Eq. (8):

$$\max_{\theta} \log p_\theta(\boldsymbol{S}^I \mid \tilde{\boldsymbol{S}}^I) = \max_{\theta} \sum_{i\in\mathcal{D}} \log p_\theta\left(\tilde{s}_i = s_i \mid \tilde{\boldsymbol{S}}^I\right) \qquad (8)$$

Let $\mathcal{D}$ denote the index set of masked tokens within the flow sequence. This approach allows us to recover the missing values of the traffic packet sequence, thereby restoring the broken sequence into an intact one.

In TFI, imputing missing data involves predicting masked tokens within raw traffic sequences. During pre-training, TFI learns packet length and traffic relationships while understanding the context of packets and sub-sequences. As shown in Fig. 5, TFI identifies the most probable tokens for missing positions, restoring sequences to restore their original integrity.

V. Experiments

A. Experiment Setup

To ascertain the efficacy and resilience of the Nüwa framework, we executed experiments across four distinct scenarios. To comprehensively evaluate the performance of the imputation models, we analyze the results by combining classification performance and Root Mean Square Error (RMSE). RMSE is used as a metric to directly assess the accuracy of imputation methods. RMSE measures the deviation between the imputed and real values, where a smaller RMSE indicates higher imputation accuracy. The formula for RMSE is as follows:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \qquad (9)$$

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

where n is the number of samples, y_i represents the actual value of the i-th sample, and $\hat{y}_i$ is the corresponding predicted value. Within each scenario, CNN, LSTM, GRU, TST, SDAE, and FS-Net were employed to evaluate the Nüwa's performance post-restoration of incomplete sequences indirectly.

Initially, we assume sufficient labeled data for training and testing in each scenario, termed source domain adaption (SDA), indicating identical source and target domains. Data in the source domain was divided into training and test sets in an 8:2 ratio. The training set was used for pre-training the TFI and training six DL models. The test set evaluated intact side-channel features, packet loss features, and imputed missing features. In a zero-shot scenario without target domain knowledge, we employed traffic from domains d_2, d_3, and d_4 to impute missing data in domain d_1, termed source-free domain adaptation (SFDA) [21]. We configure TFM strategy with ratios: Token Mask 50%, Token Deletion 10%, Span Mask 20%, Random Replace 10%, Unchanged 10%.

We trained our Nüwa using the Geforce RTX 3090. The batch size for training steps was set to 256, and the initial learning rate was set at 0.0001. Additionally, the learning rate was dynamically adjusted during pre-training.

B. Performance Improvement With TFI

1) Improvement With Source Domain Adaption: We conducted feature imputation experiments in both real-world and simulated environments. We partitioned raw traffic into training and test sets in an 8:2 ratio. The training set was utilized for pre-training and training of both the TFI (e.g., Section IV-D) and the DL models. We selected 15% of tokens from each test sample to create two sets: one replacing tokens with [MASK] for pre-training model predictions, and another with deletions for DL model testing. Pre-trained TFI predicted [MASK] positions to restore complete sequences. We then evaluated six DL models' performance with feature imputation following packet loss. The experimental results are presented in Fig. 6 and Table III. Please refer to Tables XV-XVIII, Tables XIX-XXI in Section B of [17] for detailed results.

Scenario d_1. Mobile Application Traffic Feature Imputation and Classification: As shown in Tables XV and XIX, the six DL models exhibit significant performance improvements after TFI imputation. Among them, CNN and LSTM, which are more sensitive to side-channel feature loss, achieve average performance gains of approximately 25%-26% in both real-world and simulated conditions. For instance, LSTM reaches 86.96% accuracy and 86.90% F1, approaching no-loss scenarios. From the RMSE results, TFI effectively reduces imputation errors under various packet loss rates. Specifically, for real-world data, RMSE decreases from 0.1716 (30% loss) to 0.1076 (10% loss), highlighting the robustness of TFI in mitigating feature degradation. While FS-Net and TST perform relatively well at 10% packet loss, their imputation errors grow more pronounced with higher loss rates, showing RMSE values of 0.1082 and 0.1793, respectively, at 30% loss. Overall, TFI contributes to both enhanced classification metrics and reduced RMSE, demonstrating its capability to recover lost features and improve model reliability.

Scenario d_2. Website DoH Traffic Feature Imputation and Classification: In the simulated environment, as shown in Table XVI and II, TFI improves the accuracy of six DL models by 18.16%, 27.42%, and 36.24% at 10%, 20%, and 30% packet loss rates, respectively. Similarly, F1-scores are elevated by comparable margins. In the real-world network, TFI results in an average improvement of over 26% in both accuracy and F1. Despite the performance degradation caused by packet loss, TFI enables models to maintain over 90% accuracy on average, with no more than an 8% drop at a 30% loss rate. From the RMSE perspective, TFI significantly reduces reconstruction errors under varying loss rates. Specifically, for simulated data, RMSE increases from 0.0736 at 10% loss to 0.0938 and 0.1344 at 20% and 30% loss, respectively. For real-world data, RMSE values remain comparatively low, ranging from 0.0523 (10% loss) to 0.1260 (30% loss), illustrating the robustness of TFI in preserving data integrity. These results confirm that TFI effectively mitigates the negative impact of packet loss, improving both feature reconstruction and classification performance in DoH traffic scenarios.

Scenario d_3. IoT Attack Traffic Feature Imputation and Classification: As shown in Tables XVI and XIX, missing side-channel data in IoT device traffic results in over a 14% performance decline in the models. However, TFI improves accuracy by 14.91%-22.69% and F1 by 15.18%-23.63% across six DL models. In real-world environments, TFI achieves up to a 25% accuracy improvement and a 24% F1 boost in IoT network settings. From the RMSE results, TFI significantly reduces imputation errors under varying loss rates. For simulated data, RMSE rises from 0.1403 at 10% loss to 0.2111 at 30% loss, reflecting the increasing difficulty of feature reconstruction. In real-world conditions, RMSE values are comparatively lower, ranging from 0.0746 (10% loss) to 0.1793 (30% loss), highlighting TFI's robustness in mitigating the impact of side-channel data loss. These results demonstrate that TFI effectively restores IoT traffic features and enhances both reconstruction accuracy and classification performance.

Scenario d_4. PC Terminal Feature Imputation and Classification: As shown in Tables XVIII and XXI, TFI significantly enhances the models' average performance in classifying PC flows, achieving up to 33.28% and 38.28% F1 improvements in simulated and real-world environments, respectively. From the RMSE perspective, TFI reduces reconstruction errors consistently across loss rates. For simulated data, RMSE increases from 0.1076 at 10% loss to 0.1543 at 30% loss, while for real-world data, RMSE rises from 0.0305 (10% loss) to 0.1260 (30% loss). The lower RMSE values in real-world conditions reflect TFI's ability to better reconstruct PC flow features, even under severe packet loss. This demonstrates TFI's efficacy in mitigating feature degradation and boosting classification performance in challenging network scenarios.

As illustrated in Table III and Fig. 6, packet feature loss significantly reduces DL model performance when used alone. However, with TFI, models achieve reliable classification across diverse environments by restoring missing sequence features. The S2E module ensures consistent vocabulary across real and simulated environments, avoiding out-of-vocabulary issues. Meanwhile, the TFM strategy enables the model to

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

learn contextual relationships at missing sequence positions, allowing the Model+TFI approach to effectively recover raw traffic data even under network congestion or packet loss, thereby restoring performance to near-original levels.

2) Improvement With Source-Free Domain Adaptation: Traffic capture for analysis is often impractical in real-world networks due to various constraints. Current strategies address this through Source-Free Domain Adaptation (SFDA), identifying patterns in extensive unlabeled traffic data. Studies like [21], [22], [23], and [24] use pre-trained models with unlabeled target data to improve detection under zero-shot or limited knowledge conditions. However, the core issue remains detection performance degradation from missing side-channel features due to packet loss. To assess TFI's classification robustness across network environments, we pre-train it in a zero-shot scenario using data from three scenarios, then fine-tune on the target domain to enhance downstream performance.

Table XXVIII demonstrates that the classification performance for DoH traffic can achieve over 90% accuracy even without prior domain knowledge. Additionally, an overall improvement in average accuracy and F1 of more than 12.47% and 12.73%, respectively, was observed. Fig. 18 in Section C of [17] depicts similar results.

3) Other Feature Imputation Methods: We conducted comparative experiments of our TFI against three classical raw data imputation techniques: ZeroPad, the first-order Markov chain (Markov), and three base pre-trained models, BERT, Roberta and Deberta. ZeroPad imputes missing positions with zeros directly. All of BERT, Roberta and Deberta pre-training stage employ a 15% mask ratio for input sequences, with an 80% dynamic masking and 20% no-change strategy. The comparative results are presented in Fig. 7. Please refer to Section D for detailed results.

As shown in Tables XXXI-XXXIIV of Section D in [17], TFI significantly improves the classification performance of six DL models across four scenarios. For instance, in d_2, F1-scores improve by 18.16%, 27.42%, and 36.24% on average for packet loss rates of 10%, 20%, and 30%, respectively. Similarly, flows from d_3 and d_4 also exhibit substantial F1 improvements, underscoring the consistent effectiveness of TFI. In terms of RMSE, TFI achieves the lowest values across all scenarios and loss rates, as shown in the Table XXI and Fig. 8. For example, in d_2, TFI reduces RMSE to 0.0736, 0.0938, and 0.1103 under simulated environments and 0.0523, 0.0986, and 0.1428 in real-world conditions for 10%, 20%, and 30% loss rates, respectively. Compared to other methods, TFI demonstrates a clear advantage, with RMSE values up to 60% lower than traditional methods like Markov and substantially outperforming pre-trained models such as BERT and Roberta.

These results demonstrate that TFI not only enhances feature imputation accuracy, as evidenced by lower RMSE values but also translates this improvement into higher classification performance, even under severe packet loss conditions.

Additionally, we also define an Average Deviate Ratio (ADR) to assess the deviation of feature imputation methods. Let $R_{i,v}$ denote the true value at a missing position, and $P_{i,v}$ the predicted value by imputation models. Model performance is quantified by the deviation of predictions from true values, θ_v, calculated as follows:

$$\theta_v = \frac{\sum_1^i |R_{i,v} - P_{i,v}|/R_{i,v}}{i} \tag{10}$$

As shown in Fig. 9, the TFI excels in feature imputation for bi-directions of flow, with mean errors displaying the tightest distribution at each point. In contrast, BERT and Roberta exhibits substantial prediction errors in bi-directions, escalating for smaller packet sizes. The Markov shows a convergence trend in backward prediction, shifting from larger to smaller packets, yet reaches 100% error, signifying a major divergence from true values. It exhibits an inverse trend in the forward direction, mirroring Roberta's performance on small packets. And Deberta, similar to our TFI in backward packet prediction, presents a highly erratic error distribution, with notable inaccuracies, particularly in the forward direction. Overall, our TFI achieves SOTA performance in ADR.

C. Ablation Study

To further evaluate the contributions of each module within the Nüwa framework, ablation experiments were conducted on S2E, TFM and TFI module. And additional research on sequence lengths and varying sequence mask ratios are shown on Section F of [17].

1) Analysis of TFM Module: To investigate the feature imputation capabilities of different TFM sub-strategies (e.g., Section IV-C), we designed four distinct sets of ablation experiments. These include TFM-Mask (80% MASK, 20% unchanged), TFM-Del (70% Deletion, 10% MASK, 20% unchanged), TFM-Span (80% SPAN-MASK, 20% unchanged), and TFM-RR (80% replacement, 20% unchanged). Fig. 10 and Fig. 16 (a) illustrates that the four TFM sub-strategies have similar importance weights. The average results suggest that TFM-Mask and TFM-Del play more significant roles across scenarios, with higher importance weights than TFM-Span and TFM-RR. Additional results in Section E of [17] indicate that packet loss, delay, and out-of-order issues vary in real network environments. Our TFM strategy surpasses its sub-strategies, demonstrating its superiority.

2) Analysis of S2E Module: To evaluate the contribution of the S2E module to Nüwa, we conducted a series of ablation experiments by replacing the S2E module with alternative configurations, specifically None-S2E and BPE mechanism. First, we examined the importance of the sequence embedding mechanism by completely removing the S2E module and analyzing the resultant performance. This setup, referred to as None-S2E, allows us to understand the impact of the absence of sequence embedding on the overall architecture. And then we replaced the primary design of the S2E module with a BPE mechanism, a widely used byte-level tokenizer. This substitution enables a comparative analysis to determine whether the S2E module provides any distinct advantages over more traditional tokenization methods like BPE.

As Table V and Fig. 11 shows, at a 10% loss rate, the RMSE distribution highlights the superiority of the S2E module. The model with S2E shows a highly concentrated RMSE near

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

0, indicating the highest completion accuracy. In contrast, removing S2E significantly increases RMSE and results in a more dispersed distribution. Replacing S2E with BPE slightly improves performance but still falls short of S2E. These results demonstrate that the S2E module effectively captures sequence features and enhances imputation accuracy in low-loss scenarios. Similar results are observed for other packet loss rates.

3) Analysis of TFI Module: We replaced the TFI to verify the contribution of TFI to Nüwa. First, we replaced the Transformer-Layer (self-attention mechanism) with a Feed-Forward Neural Network (RepFNN) to serve as the Non-TFI baseline. Next, we replaced the main architecture with a CNN network (RepCNN) and an LSTM network (RepLSTM), respectively. We designed comparative experiments for the three modes on scenario d_2. Please refer to Section H of [17] for detailed results.

As shown in Fig. 12, The F1 of RepFNN decreases by 5.75%-19.10% on average, while its RMSE values (0.2687-0.2977) are significantly higher than other variants, demonstrating that the self-attention mechanism provides complementary patterns beneficial for both classification and imputation tasks. Similarly, the respective decrease of 1.55%-9.40% and 1.91%-5.72% on F1 for RepCNN and RepLSTM aligns with their inferior RMSE performance (RepCNN: 0.1785-0.1991; RepLSTM: 0.1605-0.1834) compared to the complete TFI model (0.1524-0.1770). These consistent results across both metrics confirm that self-supervised tasks enhance the Transformer encoder structure's ability to learn packet side-channel feature relationships, improving both classification accuracy and imputation precision.

D. Performance Analysis of Nüwa

1) Analysis of Time Complexity: We analyzed the performance of Nüwa across 4 scenarios, focusing on two aspects: the total number of model parameters and the prediction time cost. The experimental results are shown in Table VI.First, regarding model training parameters, the total number of parameters across the 4 scenarios remains largely consistent, with minor differences attributed to variations in vocabulary sizes. Analysis of sequence length distributions reveals that in scenario d_2, approximately 1/3 of sessions exceed 512 tokens, whereas this proportion is significantly smaller in the other three scenarios. Consequently, in d_2, a higher proportion of effective data packets are lost under the same packet loss ratio, requiring more content to be imputed, which slightly slows prediction speed. In contrast, scenario d_4 has the highest proportion of sequences under 512 tokens, necessitating the least imputation and achieving the fastest prediction speed. For detailed results, please refer to Fig. 15 in Section H.

2) Analysis of Trade-Offs Between Performance and Cost: To analyze the trade-offs between model performance and computational cost, we evaluated RMSE and prediction speed across different layer configurations under various packet loss rates. As shown in Fig. 13, deeper networks generally improve RMSE but at the cost of higher computational demands. For example, the 3-layer model, with 23.75M parameters, achieves an average RMSE of 0.1972 and a prediction speed of 91 samples/s. In contrast, the 24-layer model reaches 172.56M parameters, but its RMSE worsens to 0.2182 and its speed drops to 16 samples/s, indicating overfitting risks.Among all configurations, the 12-layer structure achieves a favorable balance between performance and efficiency. It achieves the lowest overall RMSE (0.1874) across all packet loss rates, while maintaining a reasonable prediction speed of 28 samples/s. This makes it well-suited for applications that require both high precision and moderate efficiency. Meanwhile, the 6-layer model performs slightly worse in terms of RMSE (0.1897) but offers higher efficiency (58 samples per second), making it a viable alternative for scenarios with stricter latency constraints or limited computational resources.

In practical applications, the choice of model configuration depends on the specific requirements of accuracy and efficiency. For real-time or resource-constrained scenarios, a lighter configuration (3 or 6 layers) may suffice. However, for tasks that prioritize accuracy, the 12-layer model stands out as an optimal choice, offering a solid trade-off between computational cost and predictive performance. The 172M model, combined with more diverse data to prevent overfitting, is better suited for cloud-based environments with abundant computational power.

VI. Discussion

In this section, we discuss the findings and limitations of our work, as well as its potential to stimulate further research in the field.

A. Considerations of Application Domain and Limitation

The Nüwa framework imputes missing traffic data to reduce training bias and improve prediction quality. We eschew pre-training due to its impractical labeled data requirements, proposing instead that combining our TFI component with meta-learning or few-shot learning [25] can significantly improve encrypted traffic classification. Future work will expand Nüwa's current time-series imputation to include statistical features and raw data.

Nüwa necessitates substantial computational resources for training and fine-tuning, with its computational parameter count determined by the dimensionality $d = 768$, the number of layers $L = 12$, the maximum relative position $k = 512$, and three sets of additional parameters $\boldsymbol{W_{q,r}}, \boldsymbol{W_{k,r}} \in R_{d\times d}$, as well as $\boldsymbol{P} \in R^{2k\times d}$. The total number of parameters in the Nüwa framework, denoted as $N = 2L \times d^2 + 2k \times d = 88M$. This could limit its applicability in resource-constrained environments. Compared to large models with parameter counts in the hundreds of millions (B) to trillions (T), such as community-developed pre-trained models (7B, 13B) [7], [26], [27] and the GPT-4, LLaMA [28], [29] series (50B+), Nüwa's computational requirements are orders of magnitude smaller, offering substantial application potential.

B. Theoretical Performance Analysis

To theoretically analyze the performance of the pre-trained feature imputation model based on the Transformer encoder and TFM strategy, several aspects need to be considered:

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

model architecture, masking strategy, generalization ability, and computational efficiency.

Model Architecture. Nüwa (e.g., Section IV-A) enhances the Transformer's self-attention mechanism by strengthening attention to position and content, enabling it to capture long-range dependencies and learn complex features effectively. Our model's performance-cost trade-off is structurally determined, not an empirical artifact. The masking module governs this relationship, a finding confirmed by our ablation and sensitivity analyses. This trade-off's generalizability is validated by its persistence across unseen data distributions in domain adaptation experiments, establishing it as a core architectural property of the model.

TFM Strategy. Nüwa uses a sophisticated masking strategy (e.g., Section IV-C) tailored to the forms and semantic patterns of network packet loss scenarios, rather than relying on simple random masking. This approach encourages the model to learn robust feature representations, enabling it to effectively restore missing raw data in traffic, even in source-free domains. Additionally, the masking strategy in Nüwa's training process is flexible and can be configured according to the network conditions. For networks with high randomness, the Token mask ratio can be appropriately increased. In cases of severe network congestion, increasing the Span Mask ratio can enhance Nüwa's ability to handle consecutive packet loss.

Computational Efficiency. Section V-B.3 analysis reveals ZeroPad's effectiveness in practical applications. By padding packet loss locations with zeros, ZeroPad mitigates detection performance degradation with minimal computational costs. Its effectiveness stems from maintaining constant sequence length and fixed packet positions while filtering noise. Conversely, the first-order Markov model suffers from local information constraints, inability to learn from extended sequences, high resource demands, and limited imputation capability with large traffic volumes. For real-time traffic detection, ZeroPad provides an efficient, low-cost solution to counter packet loss effects on detection models.

Compound Parameter Changes. As TFI is built on the Transformer Encoder architecture, it effectively mitigates the long-distance forgetting problem, making it suitable for longer sequences. While a 15% mask ratio may not always be optimal for very long sequences, existing SOTA research [26] in NLP suggests that the mask ratio can be slightly adjusted around 15% for long text/sequence inputs. Therefore, considering the pre-training computational cost and prediction performance, a 15% mask ratio remains a stable choice in most scenarios. Similarly, for longer input sequences, the appropriate mask ratio and masking strategy can be adjusted according to the specific TFM application scenarios to better align Nüwa with practical use cases.

C. Considerations of UDP Protocol and Data Augmentation

Application in UDP. Given the prevalence of UDP in real networks, we conducted complementary experiments on UDP traffic to better characterize packet loss in mixed traffic environments. However, due to UDP's lack of a connection-oriented transmission mechanism like TCP, existing methods and tools (e.g., Wireshark) are insufficient for localizing missing UDP sequences, which posed a challenge in our analysis. As a result, in our experiments, we simulated packet loss by adding 80% of the filtered-out UDP traffic during the pre-training phase, based on the scenario outlined in Section B scenario d_1. The remaining 20% of the UDP-mutilated traffic was used in the TFI imputation phase and evaluated across six DL models. As shown in Fig. 14, the results demonstrate that Nüwa is effective for feature imputation in UDP environments.

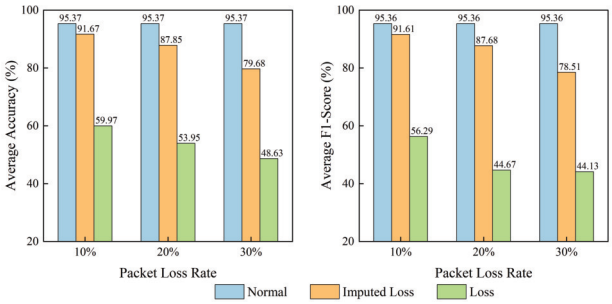


Fig. 14. Average performance in UDP feature imputation.

Data augmentation. In this paper, certain DL models underperform due to reliance solely on original time series features, lacking additional feature enhancement. Nevertheless, the significant performance improvement of these DL models is notable. Our study primarily targets the restoration of side-channel features of raw data. Hence, integrating Nüwa with additional feature extraction or fusion methods in ML/DL models could yield superior performance.

VII. Related Work

A. Traffic Packet Disturbance

Traffic packet perturbations in data center networks result from congestion and blackhole incidents, manifesting as packet loss, delays, and sequencing issues that impact application performance. Fig. 1 identifies three primary causes of traffic packet incompleteness: i) Complex topology - intricate networks with sophisticated routing create traffic bursts through multiple link aggregation and excessive node hops, causing link switching and packet loss; ii) Limited probe - computational constraints of traffic sniffers in high-performance networks lead to partial packet capture and missing session-level features; iii) Unpredictable network properties - diverse application protocols, equipment age variations, congestion, weak signals, DoS attacks, and low voltage conditions contribute to traffic data loss.

From a side-channel feature perspective, we posit that delayed packets exhibit out-of-order behavior in a time series. Prior works, such as those by Tan et al. [9], have focused on reducing packet loss through protocol optimization, developing systems to detect and locate lost packets. Li et al. introduced LossRadar [30], which captures detailed, fine-grained information on individual lost packets across networks. And Rosetta [31] conducted 1%-10% packet loss during multi-scenario traffic replay to study the differences in heterogeneous networks. While some research has aimed to locate missing

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

packets more effectively, industry inertia, with slow new protocol adoption, means little has been done to mitigate the impact of missing features on detection models during real-world packet loss events, as highlighted by You et al. [32].

B. Feature Engineering and Imputation

Feature engineering-based NTAC typically preprocesses raw data through steps such as deduplication and denoising, followed by feature extraction and representation (e.g., data augmentation) [3], [33]. The absence of certain side-channel features from encrypted traffic can result in sample deviation, thereby impacting downstream classification tasks. To address this issue, many studies adopt a "Model+" approach to preprocess input data, such as Xie et al introducing the Rosetta model to enhance traffic characterization and detection model classification as"Model + Rosetta". We categorize the impact of packet loss on side-channel features into three types, including *statistical features* (e.g., expectations, standard deviations, averages, minima, maxima, alongside packet counts and sizes, or inter-packet intervals) [34], [35], *time series features* (e.g., sequences of packet sizes, directions, and timestamps) [36], [37], and raw traffic representations (which involve encoding the raw traffic into graphical or pictorial forms) [38], [39], [40].

Statistical feature based Methods. Machine learning is extensively applied in NATC [3], with scholars achieving effective detection through statistical traffic features across time and frequency domains, complemented by machine learning. A seminal work, FlowPrint [35], introduced by Thijs et al., employs a semi-supervised approach to fingerprint mobile apps from network traffic, encrypted or otherwise, enhancing APP feature representation. Fu et al. [34] leveraged frequency domain features to represent sequential information, aiming to minimize information loss and bolster the detection model's robustness. Research [41] explores AI training traffic generation using packet payloads and statistical features. While conventional imputation methods like zero-padding, nearest-neighbor filling, and mean imputation are common, they lack contextual information and fail to capture temporal variations in missing data, limiting performance improvements.

Time Series based Methods. Side-channel features of network traffic are inherently temporal, a focus of numerous scholars [42]. Notably Zhao et al. [16] replayed network traffic and used DPDK and packet capture tools on both sender and receiver sides to analyze the impact of packet loss. Additionally, they simulated various potential scenarios using a random packet loss strategy and proposed ERNN [16] to mitigate traffic packet perturbations. TSCRNN processes raw traffic via flow segmentation and sampling, employing strategies to sample midsections of long flows for classification. These efforts aim to focus on longer flows (more than 512) [1], [11] for encrypted traffic classification. Additionally, Lin et al. [43] introduced PEAN, a multi-modal DL framework utilizing raw byte and length sequences with a self-attention mechanism to discern relationships between network packets in bi-flow. Methods for imputing missing time series features, such as Long Short-Term Memory (LSTM) networks, introduce temporal dependencies to enhance prediction accuracy. However, they still face challenges when dealing with long intervals of missing data.

Raw traffic representations. Data augmentation is prevalent in NTAC, enhancing training data diversity by transforming traffic data into images, graphs, or sentences, namely, raw traffic representations [33]. For instance, FlowPic [39] facilitates encrypted traffic classification and application identification by converting flow sequence data into images for CNN-based identification. Similarly, Kim et al. [44] and ProGraph [40] represents another approach, fusing multiple traffic features into graphs using IP nodes or ports, enhancing traffic classification with graph neural networks. Additionally, scholars like Li et al. [45] enhance data diversity and defend against analyzers using online traffic perturbation with Prism. Detection methods based on representations rely on the raw traffic data. While imputation techniques, such as SAGCRN [6], proposed a traffic data completion model based on graph convolutional network model to impute missing values. This model utilizes graph convolution to model the local spatial dependency. This work shares similarities with our approach, but it is less effective than Transformer models in handling long-distance dependencies and global information. Additionally, the graph neural network (GNN) approach requires a substantial number of labeled samples, whereas pre-training methods do not necessitate labeling. They still face significant limitations when dealing with large-scale, high-dimensional, and highly nonlinear traffic data.

C. Pre-Training Models

As previously noted, self-supervised learning is increasingly utilized for flow representation. Recently, Transformer-based pre-trained models have emerged as highly effective neural network architectures for language modeling [46]. Examples include BERT [7], RoBERTa [18], GPT [28], and BART [27]. In NATC, variants like ET-BERT [20] and PERT [47] build upon the BERT. ET-BERT pre-trains a traffic representation model on large-scale unlabeled data to produce deep contextualized datagram-level representations, fine-tunable with limited labeled data. PERT employs a transformer model for automatic traffic feature extraction using advanced dynamic word embeddings. Additionally, transformer encoders encode multimodal features for flow representation fusion [48]. Zhao et al. [8] introduced a Masked AutoEncoder (MAE) based traffic transformer with multi-level flow representation. ET-BERT capitalizes on open-domain unlabeled traffic data to learn generalized representations, similar to leveraging pre-trained outputs for downstream classification tasks.

Research related to our work falls into two categories: regular classification without missing data imputation and feature imputation across various scenarios. Our approach differs in three key ways: (i) we impute missing traffic side-channel features before applying classification, rather than performing direct classification; (ii) unlike existing feature imputation methods focused on domains like smart transportation, our work specifically addresses network traffic imputation; (iii) Nüwa framework overcomes challenges in simulating network packet loss while reducing labeling costs through unsupervised self-training.

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

VIII. Conclusion

In this paper, we present a comprehensive study spanning diverse simulated and real-world Internet scenarios, demonstrating that existing network traffic analyzers experience significant performance degradation when dealing with fragmented side-channel features. To tackle the issues, we introduce the Nüwa framework, which employs a pre-trained TFI to learn contextual traffic side-channel features through TFM strategy. Experimental results on four practical scenarios demonstrate the effectiveness and generalization of our proposed methods, which promises superior performance to the state-of-the-art methods. Extensive experiments show that TFI robustly recovers missing side-channel features across various network conditions, with imputed features aiding in enhancing the performance of current DL models.

Acknowledgment

The authors would like to thank all co-authors for their efforts.

References

[1] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2019, pp. 1171–1179.

[2] W. Bai, S. Hu, K. Chen, K. Tan, and Y. Xiong, "One more config is enough: Saving (DC)TCP for high-speed extremely shallow-buffered datacenters," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jul. 2020, pp. 2007–2016.

[3] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.

[4] Z. Wang et al., "Themis: Ambiguity-aware network intrusion detection based on symbolic model comparison," in *Proc. MTD@CCS*, Nov. 2021, pp. 31–32.

[5] J. Lin, Y. Wang, K. Kalgaonkar, G. Keren, D. Zhang, and C. Fuegen, "A time-domain convolutional recurrent network for packet loss concealment," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2021, pp. 7148–7152.

[6] Y. Zhang, X. Wei, X. Zhang, Y. Hu, and B. Yin, "Self-attention graph convolution residual network for traffic data completion," *IEEE Trans. Big Data*, vol. 9, no. 2, pp. 528–541, Apr. 2023.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, vol. 1, Jun. 2019, pp. 4171–4186.

[8] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI*, vol. 37, Jun. 2023, pp. 5420–5427.

[9] L. Tan, W. Su, W. Zhang, H. Shi, J. Miao, and P. Manzanares-Lopez, "A packet loss monitoring system for in-band network telemetry: Detection, localization, diagnosis and recovery," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4151–4168, Dec. 2021, doi: 10.1109/TNSM.2021.3125012.

[10] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, pp. 3371–3408, Mar. 2010.

[11] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 2114–2124.

[12] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark Android malware datasets and classification," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Montreal, QC, Canada, Oct. 2018, pp. 1–7.

[13] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of DoH tunnels using time-series classification of encrypted traffic," in *Proc. IEEE Int. Conf. Depend. Autonomic Secure Comput. Int. Conf. Pervasive Intell. Comput. Int. Conf. Cloud Big Data Comput. Int. Conf. Cyber Sci. Technol. Congr. (DASC/PiCom/CBDCom/CyberSciTech)*, Aug. 2020, pp. 63–70.

[14] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023.

[15] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.

[16] Z. Zhao et al., "ERNN: Error-resilient RNN for encrypted traffic detection towards network-induced phenomena," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 3, 2023, doi: 10.1109/TDSC.2023.3242134.

[17] F. Zhao. (2024). *Supplementary Material: Nvwa Experiment Results and Analysis*. [Online]. Available: https://github.com/username/repository-name

[18] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.

[19] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *Proc. ICLR*, Jan. 2020, pp. 1–21.

[20] X. Lin, G. Xiong, and G. Gou, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.

[21] S. Yang, Y. Wang, J. van de Weijer, L. Herranz, and S. Jui, "Generalized source-free domain adaptation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 8958–8967.

[22] D. Kothandaraman, S. Shekhar, A. Sancheti, M. Ghuhan, T. Shukla, and D. Manocha, "SALAD: Source-free active label-agnostic domain adaptation for classification, segmentation and detection," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2023, pp. 382–391.

[23] S. Yang, J. van de Weijer, L. Herranz, and S. Jui, "Exploiting the intrinsic neighborhood structure for source-free domain adaptation," in *Proc. NeurIPS*, vol. 34, 2021, pp. 29393–29405.

[24] S. Qu, G. Chen, J. Zhang, Z. Li, W. He, and D. Tao, "BMD: A general class-balanced multicentric dynamic prototype strategy for source-free domain adaptation," in *Proc. Eur. Conf. Comput. Vis.*, in Lecture Notes in Computer Science, Cham, Switzerland. Springer, 2022, pp. 165–182.

[25] W. Zheng, C. Gou, L. Yan, and S. Mo, "Learning to classify: A flow-based relation network for encrypted traffic classification," in *Proc. Web Conf.*, Apr. 2020, pp. 13–22.

[26] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, pp. 1–67, 2020.

[27] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 7871–7880.

[28] A. Radford and K. Narasimhan. (2018). *Improving Language Understanding by Generative Pre-Training*. [Online]. Available: https://api.semanticscholar.org/

[29] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[30] Y. Li, R. Miao, C. Kim, and M. Yu, "LossRadar: Fast detection of lost packets in data center networks," in *Proc. 12th Int. Conf. Emerg. Netw. EXperiments Technol.*, Dec. 2016, pp. 481–495.

[31] R. Xie et al., "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-aware traffic augmentation," in *Proc. ACM TUR-C*, Jul. 2023, pp. 131–132.

[32] J. You, X. Ma, D. Y. Ding, M. J. Kochenderfer, and J. Leskovec, "Handling missing data with graph representation learning," in *Proc. NeurIPS*, Jan. 2020, pp. 1–13.

[33] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2022.

[34] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.

[35] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.

[36] Z. Li, S. Chen, H. Dai, D. Xu, C.-K. Chu, and B. Xiao, "Abnormal traffic detection: Traffic feature extraction and DAE-GAN with efficient data augmentation," *IEEE Trans. Rel.*, vol. 72, no. 2, pp. 498–510, Jun. 2023.

[37] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.

[38] O. Aouedi, K. Piamrat, and D. Bagadthey, "A semi-supervised stacked autoencoder approach for network traffic classification," in *Proc. IEEE 28th Int. Conf. Netw. Protocols (ICNP)*, Oct. 2020, pp. 1–6.
[39] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
[40] W. Li, X.-Y. Zhang, H. Bao, H. Shi, and Q. Wang, "ProGraph: Robust network traffic identification with graph propagation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1385–1399, Jun. 2023.
[41] S. Hui et al., "Knowledge enhanced GAN for IoT traffic generation," in *Proc. ACM Web Conf.*, 2022, pp. 3336–3346.
[42] Q. Wen et al., "Time series data augmentation for deep learning: A survey," in *Proc. 13th Int. Joint Conf. Artif. Intell.*, Aug. 2021, pp. 4653–4660.
[43] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
[44] D. Kim, J. Han, J. Lee, H. Roh, and W. Lee, "Poster: Feasibility of malware traffic analysis through TLS-encrypted flow visualization," in *Proc. IEEE 28th Int. Conf. Netw. Protocols (ICNP)*, Oct. 2020, pp. 1–2.
[45] W. Li et al., "Prism: Real-time privacy protection against temporal network traffic analyzers," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2524–2537, 2023.
[46] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "MT-FlowFormer: A semi-supervised flow transformer for encrypted traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Min.*, 2022, pp. 2576–2584.
[47] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITUK)*, Dec. 2020, pp. 1–8.
[48] R. Zhao et al., "A novel self-supervised framework based on masked autoencoder for traffic classification," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2012–2025, Jun. 2024.



Faqi Zhao (Student Member, IEEE) received the B.S. degree from the School of Artificial Intelligence, China University of Mining and Technology, Beijing. He is currently pursuing the Ph.D. degree with the Institute of Information Engineering, Chinese Academy of Sciences, and the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include computer network security and intrusion detection.



Wenhao Li (Member, IEEE) received the B.E. degree in computer science and technology from Jinan University and the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, and the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China. He is currently a Researcher with the Research Institute, China Telecom Company Ltd. He has published more than 15 papers in network security in international journals and conference proceedings, including CCS, IEEE TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, COMNETS, HPCC, and ICCS. His research interests include computer network security, cybersecurity, and AI security.



Huaifeng Bao (Member, IEEE) received the B.S. degree in information and computational sciences from Peking University and the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, and the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include computer network security and malware detection.



Zhaoxuan Li (Member, IEEE) is currently pursuing the Ph.D. degree with the State Key Laboratory of Information Security (SKLOIS), Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), Beijing, China. He has published more than 30 papers in international journals and conference proceedings, including IEEE TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, COMNETS, COSE, ESE, INFOCOM, SIGCOMM, WWW, CCS, and ICWS. His research interests include traffic identification, blockchain security, formal methods, and privacy-preserving.



Guoqiao Zhou is currently an Engineer with the Institute of Information Engineering, Chinese Academy of Sciences. He has published several academic articles in the field of cyber security and have extensive practical experience. His main research interests are network traffic measurement and analysis, and network security.



Wen Wang is currently an Associate Researcher with the Institute of Information Engineering, Chinese Academy of Sciences (IIEC), and a member of the CAS Youth Innovation Promotion Association, with major research interests include advanced threat discovery, encrypted traffic analysis, and network countermeasures.



Feng Liu is currently a Researcher with the Institute of Information Engineering, Chinese Academy of Sciences (CAS), and a Professor and Ph.D. Supervisor with the School of Cyberspace Security, University of Chinese Academy of Sciences (UCAS). His main research interests include cyber countermeasures, cyberspace security strategies, and cryptography.

Authorized licensed use limited to: UNIVERSITY OF BIRMINGHAM. Downloaded on November 09,2026 at 13:21:24 UTC from IEEE Xplore. Restrictions apply.